

Katana Platform — Architecture & Module Connections

Document version: June 2026

Audience: Internal team (product, engineering, sales, customer success)

Purpose: How Katana modules connect, synchronize, and adapt to each client's plan.

Executive summary

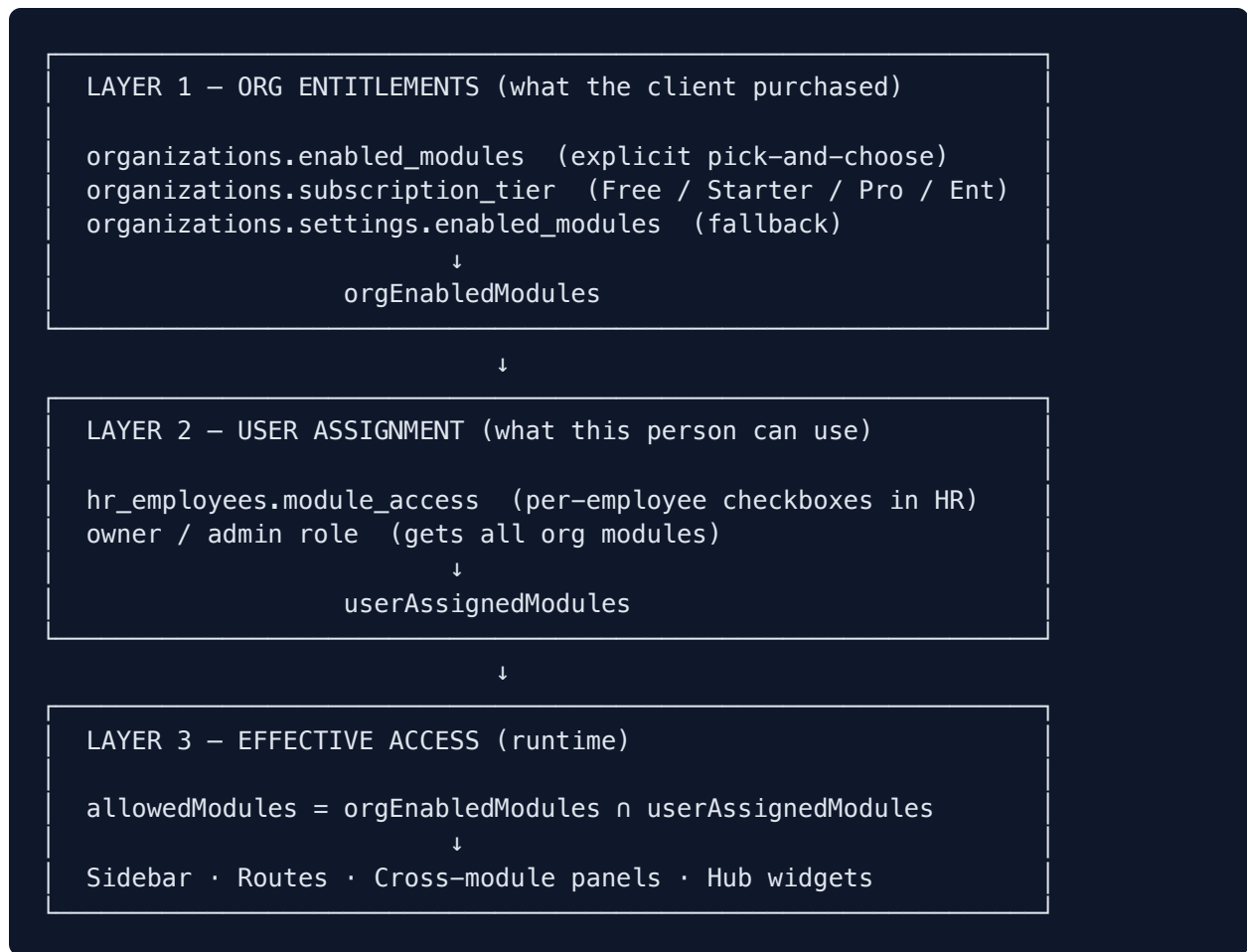
Katana is a **modular business platform** with **15 registered modules**. Clients can run the full enterprise stack or pick and choose modules. The system:

1. Knows **what modules the organization has** (org entitlements)
2. Knows **what each user can access** (HR module assignment)
3. **Automatically wires** cross-module connections only where both modules exist
4. **Aggregates everything** into the Hub as mission control

One line for leadership: Add a module → connections appear. Remove a module → UI hides cleanly with no broken links.

1. Access control — three layers

Everything flows through a gating stack before any cross-module feature appears.



Rules:

Question	Answer
Does this integration exist for the org?	Both modules must be in <code>orgEnabledModules</code>
Can this user use the integration?	Both modules must be in <code>allowedModules</code>
Can HR assign a module to an employee?	Only modules in <code>orgEnabledModules</code>

Source files: `src/lib/org-module-access.ts` , `src/contexts/ModuleAccessContext.tsx`

2. All 15 modules

ID	Product name	Route
<code>hub</code>	Hub	<code>/hub</code>

ID	Product name	Route
projects	Katana PM	/projects
inventory	Katana Inventory	/inventory
customer-success	Katana Customers (CRM + KYC)	/customer-success
workforce	WFM	/workforce
hr	Katana HR	/hr
employee	Employee Portal	/employee
careers	Careers	/careers
manufacturing	Katana Facilities	/manufacturing
automation	Automation (Katana Sync)	/automation
kyi	Know Your Investor	/kyi
comms	Katana Comms	/comms
agents	Agent Office	/agents
support	Katana Support	/support
finance	Katana Finance	/finance

Source file: `src/lib/module-access.ts`

3. Plan tiers — default module bundles

Tier	Modules included	Typical use case
Free	Hub, Customers, Employee Portal, Support	Core suite trial
Starter	Free + PM, HR, Careers	Small team getting started
Professional	Starter + WFM, Inventory, Comms, Finance, Automation	Full operations stack

Tier	Modules included	Typical use case
Enterprise	All 15 modules	Full platform
Custom	<code>enabled_modules</code> array in database	Pick-and-choose à la carte

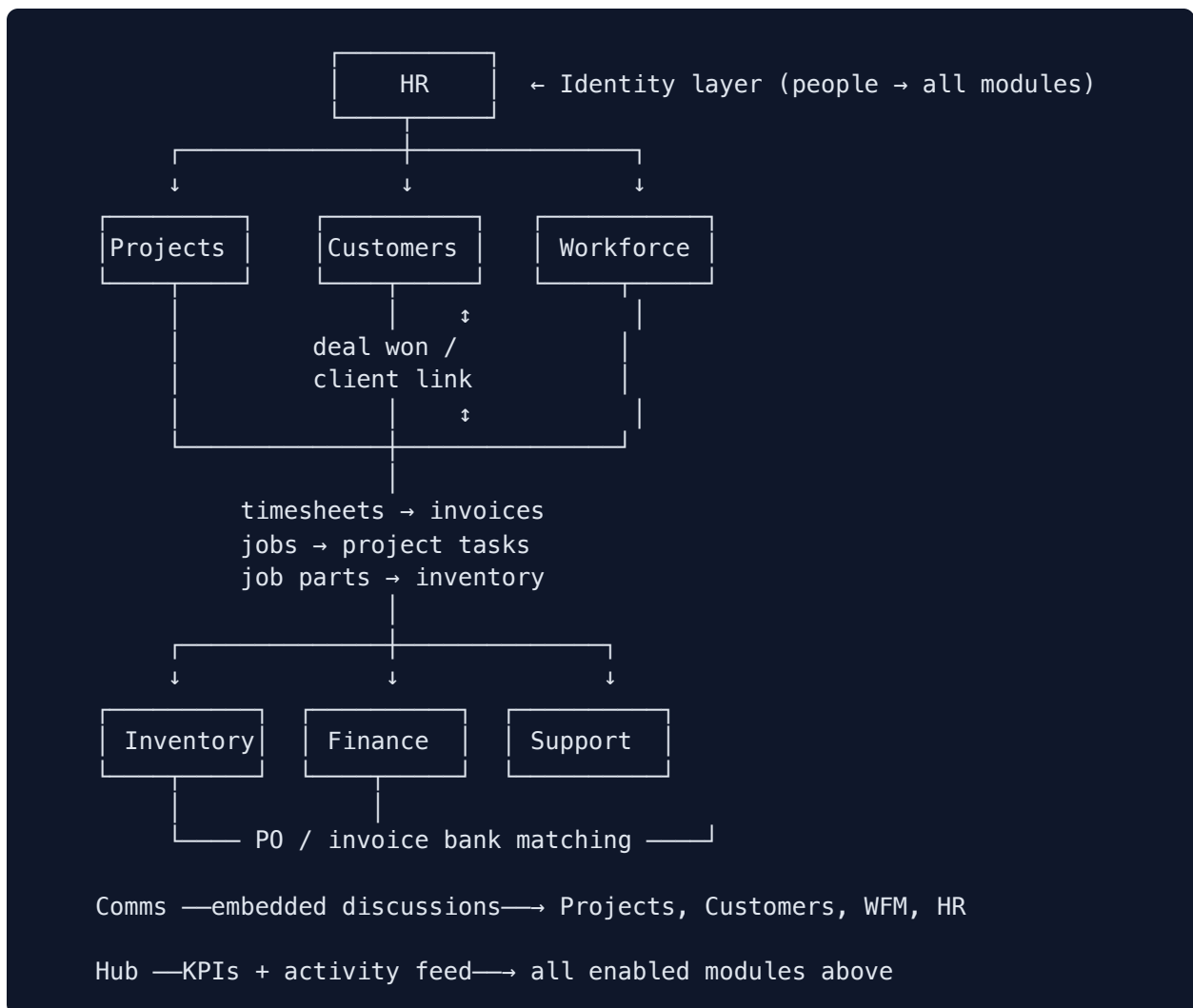
Core suite (included on every plan): Hub, Customers, Employee Portal, Support.

Source file: `src/lib/module-bundles.ts`

4. Integration graph — how modules talk to each other

Connections only activate when **both modules** are in the org's entitlement list.

4.1 Visual map



4.2 Connection reference table

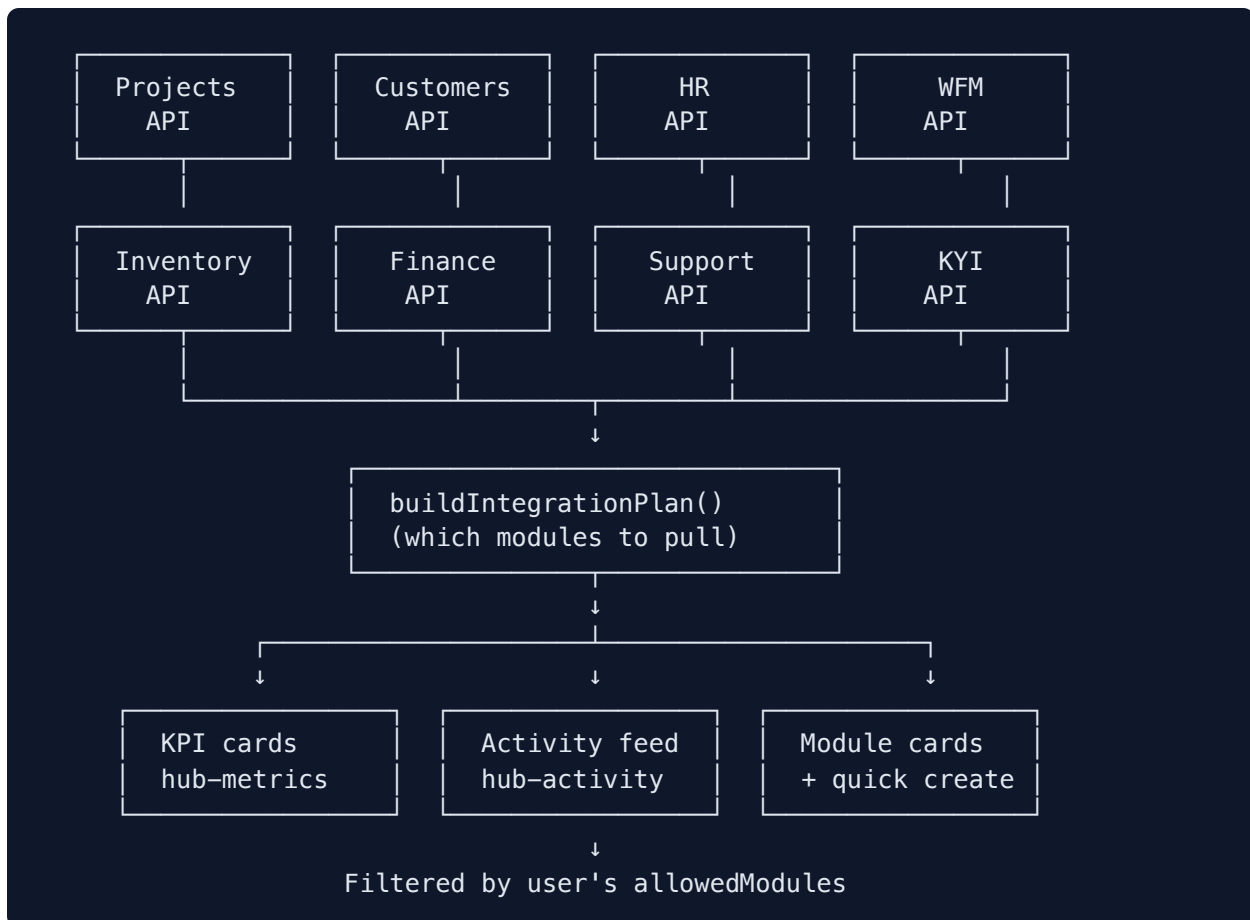
From	To	What syncs	Implementation
Customers	WFM	Client on work order; deal won → work order	<code>crm-workflows.ts</code> , <code>wfm-integrations.ts</code>
WFM	Customers	Approved timesheets → draft invoice	<code>createDraftInvoiceFromJob()</code>
WFM	Projects	<code>project_id</code> , <code>task_id</code> on jobs	<code>wfm-integrations.ts</code> , <code>PmProjectLinkedRecords</code>
WFM	Inventory	Job parts; checkout on completion	<code>wfm_job_parts</code> , <code>checkoutJobPartsOnCompletion()</code>
WFM	Finance	Billable time → receivables path	WFM panel + Finance matching
Customers	Finance	Invoices ↔ bank deposit matching	<code>finance-match-suggestions.ts</code>
Inventory	Finance	POs ↔ bank outflow matching	<code>finance-match-suggestions.ts</code>
Customers	Support	<code>client_id</code> on support tickets	<code>customer-linked-records.ts</code>
HR	Projects	Employee assignees on tasks	<code>notification-recipients.ts</code>
HR	WFM	Technician roster	<code>wfm-api.ts</code>
HR	Customers	CSM → auth user link	<code>notification-recipients.ts</code>
Comms	Projects	Task discussion threads	<code>ModuleDiscussion</code> component
Comms	Customers	Client discussion threads	<code>ModuleDiscussion</code> component
Comms	WFM	Job discussion threads	<code>ModuleDiscussion</code> component

From	To	What syncs	Implementation
Comms	HR	Employee discussion threads	<code>ModuleDiscussion</code> component
Hub	All enabled	KPIs, activity feed, quick search/create	<code>hub-metrics.ts</code> , <code>hub-activity.ts</code>

Source file: `src/lib/module-integrations.ts`

5. Hub — the synchronization layer

Hub does not own data. It **aggregates** from every module the org has enabled, then **filters** what each user can see.



Hub aggregates from: Projects, Customers, HR, Careers, Inventory, WFM, Support, KYI, Finance, Facilities, Automation.

Source files: `src/pages/HubPage.tsx` , `src/lib/hub-metrics.ts` , `src/lib/hub-activity.ts`

6. Shared platform services (glue)

These run across modules and are not separate sidebar items:

Service	Role	Key file
HR identity	Maps employees → auth users for assignments & notifications	<code>notification-recipients.ts</code>
Notifications	Event bus per module (PM, HR, CS, WFM, Finance, etc.)	<code>notification-modules.ts</code>
Comms context links	Messages linked to client / project / task / job / employee	<code>comms-api.ts</code> , <code>ModuleDiscussion.tsx</code>
Storage	File buckets per module (projects, hr, inventory, wfm, customers)	<code>storage-api.ts</code>
Katana Sync / Agents	AI context per module	<code>ksync-context.ts</code> , Agent Office

7. Example scenarios

Scenario A — Client has Customers + WFM only

- Deal won in CRM → creates work order in WFM ✓
- Timesheet on job → draft invoice in Customers Commerce ✓
- Finance bank matching links → hidden (no Finance module)
- Inventory parts on jobs → hidden (no Inventory module)
- Hub shows Customers + WFM KPIs only

Scenario B — Professional plan (no KYI, Agents, Facilities)

- Full CS ↔ WFM ↔ PM ↔ Inventory ↔ Finance ↔ Comms stack ✓
- Hub shows all professional-tier KPIs ✓
- KYI, Agents, Facilities not in sidebar

Scenario C — Enterprise

- All 15 modules, all integrations active
- Hub = full mission control

Scenario D — User has WFM but not Customers

- Can see their jobs
- Client name visible on job (read-only)
- Cannot open Customers tab or create invoices (no CS access)
- Manager with both modules sees full link

8. Where to view this in the app

Location	What it shows
Settings → Organization	"Modules & connections" card — enabled modules, active integrations, unlock hints
Hub	Live KPIs and activity from connected modules
Customer detail	Linked WFM jobs, invoices, support tickets
Project detail	Linked WFM jobs
WFM job panel	Cross-module links (invoice, project, parts)
HR → Employee profile	Module access assignment (clamped to org entitlements)

9. Database & configuration

Item	Table / column	Purpose
Org module list	<code>organizations.enabled_modules</code>	Explicit pick-and-choose entitlements
Plan tier	<code>organizations.subscription_tier</code>	Default bundle when <code>enabled_modules</code> is null

Item	Table / column	Purpose
User modules	<code>hr_employees.module_access</code>	Per-user sidebar access
Comms links	<code>comms_context_links</code>	Discussion context types: task, client, employee, project, job, invoice

Migrations to run in Supabase:

- `supabase-org-modules-migration.sql` — org entitlements column
- `supabase-comms-context-expansion-migration.sql` — job + invoice comms contexts
- `supabase-finance-notifications-migration.sql` — finance notification source
- `supabase-localhost-owner-full-modules.sql` — restore full access for dev owners

10. Engineering reference — key source files

Concern	Path
Module registry	<code>src/lib/module-access.ts</code>
Tier bundles	<code>src/lib/module-bundles.ts</code>
Org entitlements resolver	<code>src/lib/org-module-access.ts</code>
Integration graph	<code>src/lib/module-integrations.ts</code>
User access context	<code>src/contexts/ModuleAccessContext.tsx</code>
Integration hook	<code>src/hooks/use-module-integration.ts</code>
WFM cross-links	<code>src/lib/wfm-integrations.ts</code>
CRM workflows	<code>src/lib/crm-workflows.ts</code>
Finance matching	<code>src/lib/finance-match-suggestions.ts</code>
Customer linked records	<code>src/lib/customer-linked-records.ts</code>
Hub metrics	<code>src/lib/hub-metrics.ts</code>
Hub activity	<code>src/lib/hub-activity.ts</code>

Concern	Path
Org settings UI	src/components/settings/OrgModulesIntegrationCard.tsx

11. Mermaid diagrams (for presentations)

Paste any block below into <https://mermaid.live> to export as PNG/SVG for PowerPoint or Word.

Access control flow

```
flowchart TB
    subgraph provisioning [Org entitlements]
        DB[enabled_modules]
        Tier[subscription_tier]
        DB --> OrgMods[orgEnabledModules]
        Tier --> OrgMods
    end
    subgraph users [User assignment]
        HR[module_access]
        Role[owner/admin]
        HR --> UserMods[userAssignedModules]
        Role --> UserMods
    end
    subgraph effective [Runtime]
        Intersect["allowedModules = org n user"]
        OrgMods --> Intersect
        UserMods --> Intersect
        Intersect --> UI["Sidebar + integrations + Hub"]
    end
```

Module integration graph

```
flowchart TB
    HR[HR]
    PM[Projects]
    CS[Customers]
    WFM[WFM]
    INV[Inventory]
    FIN[Finance]
    SUP[Support]
    COM[Comms]
    HUB[Hub]

    HR --> PM
    HR --> WFM
    HR --> CS
    CS <--> WFM
    WFM --> PM
    WFM --> INV
    WFM --> FIN
    CS --> FIN
    INV --> FIN
    CS --> SUP
    COM -. -> PM
    COM -. -> CS
    COM -. -> WFM
    COM -. -> HR
    HUB --> PM
    HUB --> CS
    HUB --> WFM
    HUB --> INV
    HUB --> FIN
    HUB --> SUP
```

Katana vv2 — Internal architecture document. Update when new modules or integration edges are added to `module-integrations.ts`.